

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ федерального государственного бюджетного
образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА»
Техникум Пермского института (филиала)

Практическая работа №1
по дисциплине «Поддержка и тестирование программных модулей»
Тема: «Разработка приложения управления складами»
Студента группы ИПо-32
Овчинникова Кира

2026 г.

Оглавление

ВВЕДЕНИЕ	3
ГЛАВА 1. НАЗНАЧЕНИЕ И ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ	4
ГЛАВА 2. СТРУКТУРА ПРОЕКТА.....	4
ГЛАВА 3. МОДЕЛИ ДАННЫХ	6
3.1. Product	6
3.2. Warehouse	6
3.3. Organization	7
3.4. Invoice	7
3.5. Category	7
3.6. Manufacturer	7
3.7. Supplier	7
ГЛАВА 4. РЕАЛИЗАЦИЯ ФУНКЦИОНАЛА.....	8
4.1. Управление товарами	8
4.2. Управление складами	9
4.3. Работа с накладными.....	9
4.4. Импорт CSV	10
ГЛАВА 5. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС.....	12
ГЛАВА 6. РАЗРАБОТКА И ТЕСТИРОВАНИЕ.....	12
6.1. Этапы разработки	12
6.2. Тестирование	12
ЗАКЛЮЧЕНИЕ	13

ВВЕДЕНИЕ

Проект WarehouseApp создан для автоматизации учета товаров и управления складами в организациях. Цель приложения — предоставить удобный инструмент для контроля остатков, оформления накладных на приход и расход товаров, управления организациями и складами, а также импорта данных.

Приложение разработано на языке C# с использованием платформы .NET 10 и графического интерфейса WPF. Интерфейс построен на основе окон (Views), а логика частично реализована через сервис DataService, часть в коде окон, что отражает частичное применение паттерна MVVM.

В рамках проекта также написаны модульные тесты на xUnit, которые проверяют корректность работы бизнес-логики и целостность данных.

ГЛАВА 1. НАЗНАЧЕНИЕ И ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ

WarehouseApp предназначен для учета товаров на складах и управления организационными структурами.

Программа предоставляет возможности:

- Управлять организациями: добавлять, редактировать, удалять;
- Управлять складами: создавать, редактировать и удалять с проверкой связанных товаров;
- Управлять товарами: добавлять, редактировать и удалять, связь с конкретным складом;
- Формировать накладные на приход и расход товаров, автоматически обновляя количество на складе;
- Импортировать данные из CSV-файлов для быстрого добавления товаров на выбранный склад;
- Просматривать актуальное состояние складов и товаров через таблицы.

ГЛАВА 2. СТРУКТУРА ПРОЕКТА

Проект состоит из следующих компонентов:

- Models — описывают все данные системы. Включают модели: Product, Warehouse, Organization, Invoice и другие вспомогательные объекты. Каждая модель содержит свойства, необходимые для работы бизнес-логики, включая идентификаторы, названия, адреса, количество товаров, цену и скидки.

— Services — сервисный слой DataService, управляющий коллекциями объектов и выполняющий все CRUD-операции, обработку накладных и импорт CSV. Этот слой обеспечивает целостность данных и возможность модульного тестирования.

— Views — графический интерфейс WPF. Окна включают:

1. StorageWindow — главное окно с таблицей товаров и списком складов;
2. ProductWindow — окно редактирования товаров;
3. WarehouseWindow — окно редактирования складов;
4. InvoiceWindow — окно оформления накладных;

Интерфейс позволяет пользователю выполнять все операции с объектами системы и видеть изменения в реальном времени.

— Tests — проект с модульными тестами на xUnit, проверяющими корректность бизнес-логики.

Часть логики реализована напрямую в коде окон, часть через сервис DataService. MVVM используется частично: данные связаны с элементами интерфейса через свойства и коллекции, но события частично обрабатываются в коде окон.

ГЛАВА 3. МОДЕЛИ ДАННЫХ

3.1. Product

Свойства модели Product:

- Id — уникальный идентификатор товара;
- WarehouseId — связь со складом;
- Name — название товара;
- Category — категория товара;
- Manufacturer — производитель;
- Supplier — поставщик;
- Price — цена;
- StockQuantity — количество на складе;
- DiscountPercent — размер скидки.

Продукт связан с конкретным складом и используется в накладных, таблицах и импорте CSV.

3.2. Warehouse

Свойства модели Warehouse:

- Id — уникальный идентификатор;
- OrganizationId — связь с организацией;
- Name — название склада;
- Address — адрес склада.

Склад содержит список товаров, привязанных через WarehouseId, и участвует в накладных

3.3. Organization

Свойства модели Organization:

- Id — уникальный идентификатор;
- Name — название организации.

Организация может иметь несколько складов и управлять ними.

3.4. Invoice

Накладные фиксируют операции прихода и расхода товаров:

- Содержат список товаров и количество;
- Автоматически обновляют StockQuantity на складе;
- Контролируют корректность операций, чтобы не допустить отрицательного количества;
- Применяются через окно InvoiceWindow.

3.5. Category

Модель Category описывает тип товара и используется для группировки и фильтрации товаров.

3.6. Manufacturer

Модель Manufacturer содержит данные о компаниях, производящих товары, и связывается с продуктами через поле Manufacturer.

3.7. Supplier

Модель Supplier содержит информацию о компаниях-поставщиках и связывается с продуктами через поле Supplier.

ГЛАВА 4. РЕАЛИЗАЦИЯ ФУНКЦИОНАЛА

4.1. Управление товарами

Пользователь может управлять товарами через главное окно приложения или окно редактирования ProductWindow.

Основные возможности:

1. Добавление товара:

- Пользователь выбирает склад и нажимает «Добавить товар».
- Открывается окно ProductWindow, где вводятся свойства товара: название, категория, производитель, поставщик, цена, количество и скидка.
- После подтверждения данные сохраняются в коллекцию dataService.Products, автоматически присваивается уникальный идентификатор.

- Таблица товаров обновляется и отображает новый объект.

2. Редактирование товара:

- Пользователь выбирает товар в таблице и нажимает «Изменить».
- Открывается окно ProductWindow с заполненными текущими значениями.
- После изменения данных они сохраняются обратно в коллекцию и таблица обновляется.

3. Удаление товара:

- Пользователь выбирает товар в таблице и нажимает «Удалить»
- Товар удаляется из коллекции Products и таблица автоматически обновляется.
- Обеспечивается контроль зависимостей, чтобы удаление не нарушало целостность данных (например, связь с накладными).

4. Отображение данных:

- Таблица товаров показывает все важные свойства: название, категорию, производителя, поставщика, цену, количество и скидку.

— Обновление таблицы происходит автоматически после добавления, редактирования или удаления.

4.2. Управление складами

Склады управляются через StorageWindow и отдельное окно WarehouseWindow.

Основные возможности:

1. Создание нового склада:

- Ввод названия и адреса склада.
- Связь с организацией через OrganizationId.
- После создания склад добавляется в коллекцию

dataService.Warehouses и отображается в списке складов.

2. Редактирование склада:

— Пользователь выбирает склад, редактирует данные и сохраняет изменения.

- Изменения обновляют коллекцию и список складов.

3. Удаление склада:

— Пользователь выбирает склад, нажимает удаление и подтверждает.

— Все товары, связанные со складом, удаляются автоматически для сохранения целостности.

— Система предотвращает потерю данных и обеспечивает согласованность коллекций.

4.3. Работа с накладными

Накладные оформляются через окно InvoiceWindow. Они разделяются на входящие и исходящие

1. Создание накладной:

- Пользователь выбирает склад и товары для накладной.
- Указывается количество для каждой позиции.

- Выбирается тип накладной: входящая (поступление) или исходящая (списание)

2. Применение накладной:

- После подтверждения накладной изменяется StockQuantity каждого выбранного товара.

- Обновляется таблица товаров в главном окне, чтобы пользователь видел актуальные остатки.

3. Контроль корректности:

- Проверяется, чтобы остатки не стали отрицательными.

- В случае ошибки отображается предупреждение пользователю.

- Обеспечивается надежность учета товаров и предотвращение ошибок при расходе или приходе.

4.4. Импорт CSV

В приложении WarehouseApp реализована функция импорта товаров из CSV-файлов, что позволяет быстро добавлять большое количество товаров на выбранный склад без ручного ввода. Эта возможность особенно полезна, если организация регулярно получает новые партии товаров и нужно оперативно обновлять данные о складе.

Основные этапы работы функции:

1. Выбор файла пользователем:

- Пользователь выбирает склад, на который будут добавлены товары.

- Нажимает кнопку «Импорт» на панели инструментов.

- Открывается диалоговое окно выбора файла, где можно указать путь к файлу формата CSV.

- Фильтр файла настроен на отображение только CSV (*.csv), но можно выбрать и все файлы.

2. Проверка формата и структуры файла:

— Система проверяет наличие всех обязательных колонок: название товара, категория, производитель, поставщик, цена, количество на складе, размер скидки.

— Для числовых значений (Price, StockQuantity, DiscountPercent) выполняется проверка корректного формата.

— Пустые строки или строки с некорректными данными игнорируются, чтобы не нарушить целостность коллекций.

3. Обработка данных и создание объектов Product:

1. Каждая строка CSV преобразуется в объект Product.

2. Присваивается уникальный идентификатор Id.

3. Связь с выбранным складом устанавливается через WarehouseId.

4. Товары добавляются в коллекцию dataService.Products.

4. Автоматическое обновление интерфейса:

— После завершения импорта таблица товаров в StorageWindow автоматически обновляется.

— Статусная строка информирует пользователя о результате операции: «Импорт товаров завершён!»

5. Обработка ошибок и уведомления:

— Если обнаружена ошибка в формате данных, система выводит всплывающее сообщение с указанием причины.

— Пользователь может исправить файл и повторить импорт без нарушения работы приложения.

ГЛАВА 5. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

Главное окно StorageWindow включает:

- Меню управления файлами, складами, товарами, накладными;
- Список складов и таблицу товаров;
- Кнопки добавления, редактирования и удаления объектов;
- Статусную строку для отображения текущего контекста (организация и склад).

Навигация: сначала выбирается организация, затем склад, после чего пользователь получает доступ к товарам и накладным.

ГЛАВА 6. РАЗРАБОТКА И ТЕСТИРОВАНИЕ

6.1. Этапы разработки

1. Создание моделей данных для товаров, складов, организаций, категорий, производителей, поставщиков и накладных;
2. Реализация сервиса DataService для работы с коллекциями;
3. Разработка интерфейса на WPF с окном Storage, окнами редактирования товаров, складов и накладных;
4. Добавление функций накладных и импорта CSV;
5. Написание модульных тестов на xUnit.

6.2. Тестирование

Тесты проверяют:

1. Добавление, редактирование и удаление организаций, складов, товаров, категорий, производителей и поставщиков;
2. Применение накладных на приход и расход товаров;
3. Корректность изменения количества товаров на складе;
4. Импорт CSV;
5. Согласованность связей между объектами.

Все тесты прошли успешно.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного проекта был разработан полностью функциональный программный комплекс WarehouseApp, предназначенный для управления складскими операциями и учета товаров в организациях. Работа над проектом позволила реализовать все заявленные требования и функциональные возможности, включая:

- Учет товаров с указанием категории, производителя, поставщика, цены, количества и размера скидки;
- Управление складами и организациями с возможностью создания, редактирования и удаления объектов;
- Работу с накладными, позволяющую оформлять приход и расход товаров партиями с автоматическим обновлением остатков на складе;
- Импорт данных из CSV-файлов для быстрого добавления большого количества товаров на выбранный склад;
- Тестирование бизнес-логики через xUnit, включая проверки CRUD-операций, накладных и корректности изменения остатков;

Приложение построено на платформе .NET 10 с использованием WPF, что обеспечивает удобный и интуитивно понятный интерфейс для пользователя. Частичное применение MVVM позволило разделить данные и представление, а сервисный слой DataService обеспечил надежное управление коллекциями и поддержку связей между объектами.

Особое внимание было уделено контролю целостности данных и обработке ошибок: пользователь получает уведомления при некорректном вводе, предотвращается создание отрицательных остатков и соблюдаются все зависимости между моделями.

Проект демонстрирует навыки комплексной разработки: от проектирования моделей данных до реализации интерфейса и тестирования. Кроме того, архитектура приложения позволяет в будущем расширять функциональность — добавлять базу данных, отчеты, авторизацию пользователей и дополнительные фильтры для аналитики.

Таким образом, WarehouseApp является завершенным и стабильным приложением, пригодным для практического использования в задачах управления складом и учета товаров, а также служит хорошей демонстрацией применения технологий C#, WPF, коллекций данных и модульного тестирования.